

```
[69]: #기상데이터 분석 실습
#특정 지역의 1년치 기상 데이터를 담은 csv 파일을 활용하여
#다음날 비가 오지않는지에 대한 예측을 할 수 있는 의사결정나무 모델을 만드시오
#Random Forest 사용
```

```
[70]: # 라이브러리 불러오기
import pandas as pd
import numpy as np
from sklearn import model_selection
from sklearn import metrics
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
[71]: # 데이터프레임으로 데이터 불러오기
df = pd.read_csv("data/weather.csv")
print(df.shape)
df.head()

# 전처리
df = df.drop(['WindGustDir', 'WindDir9am', 'WindDir3pm'], axis=1)

# sunshine, windgustspeed, windspeed9am의 결측치인 값을 삭제
df = df.dropna(subset=['Sunshine', 'WindGustSpeed', 'WindSpeed9am'])

# 전처리: 인위형 변수로 변경
df["RainToday"] = df.RainToday.map({"No":0, "Yes":1})
df["RainTomorrow"] = df.RainTomorrow.map({"No":0, "Yes":1})

# Input 변수(X)와 Output 변수(y) 구분
X = np.array(df.iloc[:, :-1])
y = np.array(df['RainTomorrow'])

# 트레이닝 셋과 테스트 셋으로 데이터 구분(여기서는 7:3으로 구분)
from sklearn.model_selection import train_test_split

# random_state는 반복적으로 같은 결과를 내기 위해서 설정
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

(365, 21)
```

```
[72]: # 의사결정나무모델에 데이터 적합(fitting)
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=0) #max_depth 0/지정
tree.fit(X_train, y_train)
```

```
[72]: + DecisionTreeClassifier
DecisionTreeClassifier(random_state=0)
```

```
[73]: #변수의 중요도(y값 결정에 대한 기여 정도)를 변수로 수치화하여 보여줌
tree.feature_importances_
```

```
[73]: array([[0.12229192, 0.06376438, 0.00756904, 0.08510398, 0.10892539,
0.00779166, 0.04700437, 0.02355307, 0.10899239, 0.03383806,
0.0769625 , 0.26846222, 0.          , 0.04574103, 0.          ,
0.          , 0.          ]])
```

```
[74]: pd.DataFrame(tree.feature_importances_.reshape(1,-1), index=['feature_importance'])
```

```
[74]:
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
feature_importance	0.122292	0.063764	0.007569	0.085104	0.108925	0.007792	0.047004	0.023553	0.108992	0.033838	0.076962	0.268462	0.0	0.045741	0.0	0.0

```
[75]: df.columns

pd.DataFrame(tree.feature_importances_.reshape(1,-1),
              columns=df.columns[1:], index=['feature_importance'])

#보기 좋게 컬럼명을 바꿈
```

```
[75]:
```

	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustSpeed	WindSpeed9am	WindSpeed3pm	Humidity9am	Humidity3pm	Pressure9am	Pressure3pm	Cloud9am
feature_importance	0.122292	0.063764	0.007569	0.085104	0.108925	0.007792	0.047004	0.023553	0.108992	0.033838	0.076962	0.268462

```
[76]: #변수중요도를 데이터프레임으로 만들기

importance_df = pd.DataFrame({'feature': df.columns[1:], 'importance': tree.feature_importances_})
importance_df
```

```
[76]:
```

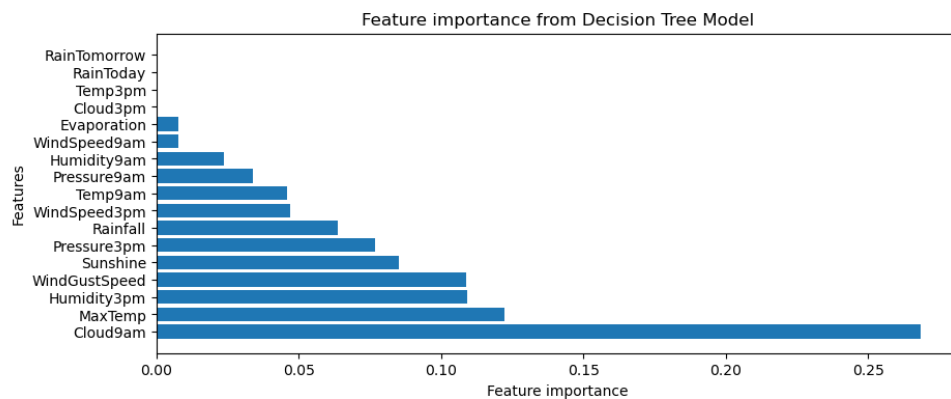
	feature	importance
0	MaxTemp	0.122292
1	Rainfall	0.063764
2	Evaporation	0.007569
3	Sunshine	0.085104
4	WindGustSpeed	0.108925
5	WindSpeed9am	0.007792
6	WindSpeed3pm	0.047004
7	Humidity9am	0.023553
8	Humidity3pm	0.108992
9	Pressure9am	0.033838
10	Pressure3pm	0.076962
11	Cloud9am	0.268462

```
[77]: #영향을 아예 끼치지 않는 Cloud3pm, Temp3pm을 삭제.
# RainToday도 0이지만 개인 판단으로 삭제하지 않음
```

```
[78]: #제거하기
df = df.drop(['Cloud3pm', 'Temp3pm'], axis=1)
```

```
[79]: #중요도에 따라 내림차순으로 정렬
importance_df = importance_df.sort_values(by='importance', ascending=False)

import matplotlib.pyplot as plt
#막대 그래프로 시각화
plt.figure(figsize=(10,4))
plt.barh(importance_df['feature'], importance_df['importance'])
plt.xlabel('Feature importance')
plt.ylabel('Features')
plt.title('Feature importance from Decision Tree Model')
plt.show()
```



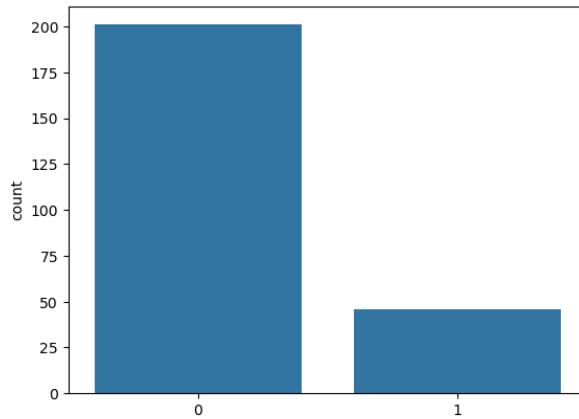
```
[80]: #Cloud9am이 가장 중요한 것을 직관적으로 볼 수 있다.
```

```
[81]: #데이터에 불균형이 있었는지 확인
```

```
import matplotlib.pyplot as plot
import seaborn as sns

sns.countplot(x=y_train)
#밸런싱 비교는 train 할 거고, test는 손 안됨. y데이터 확인 = y_train
#꼭 그래프를 그려야 하는 건 아님. value_counts()로 확인해도 됨
```

```
[81]: <Axes: ylabel='count'>
```



```
[82]: #데이터를 부분
```

```
from imblearn.over_sampling import SMOTE

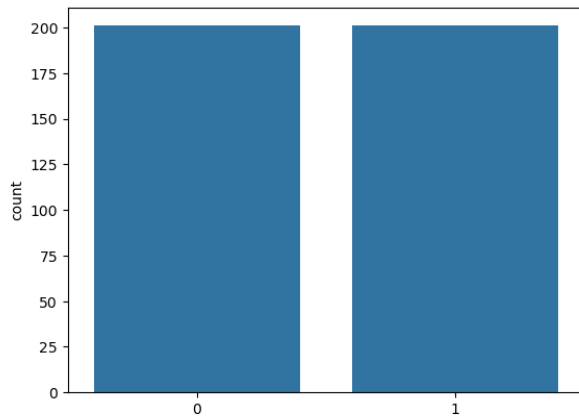
sm = SMOTE(random_state=0)
X_train_balanced, y_train_balanced = sm.fit_resample(X_train, y_train)

print("X_train_balanced 크기:", X_train_balanced.shape)
print("y_train_balanced 크기:", y_train_balanced.shape)

X_train_balanced 크기: (402, 17)
y_train_balanced 크기: (402,)
```

```
[83]: sns.countplot(x=y_train_balanced)
```

```
[83]: <Axes: ylabel='count'>
```



```
[84]: train_scores, test_scores = list(), list()
# train_score와 test_score의 정확도를 저장 장소
# train_score는 훈련데이터에 특화될수록 증가
# test_score는 테스트데이터에 일반화될수록 증가
```

```
[85]: # 정확도 계산
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

# 의사결정나무모델에 데이터 적합(fitting)
from sklearn.tree import DecisionTreeClassifier

# 의사결정나무 모델의 depth 조절(1부터 19까지)
for i in range(1,20):
    df = DecisionTreeClassifier(max_depth=i)
    # 깊이를 증가시키면서 의사결정나무 모델 적합
    df.fit(X_train, y_train)

    #training dataset 정확도 평가
    train_att = df.predict(X_train)
    train_acc = accuracy_score(y_train, train_att)
    train_scores.append(train_acc)

    #test dataset 정확도 평가
    test_att = df.predict(X_test)
    test_acc = accuracy_score(y_test, test_att)
    test_scores.append(test_acc)

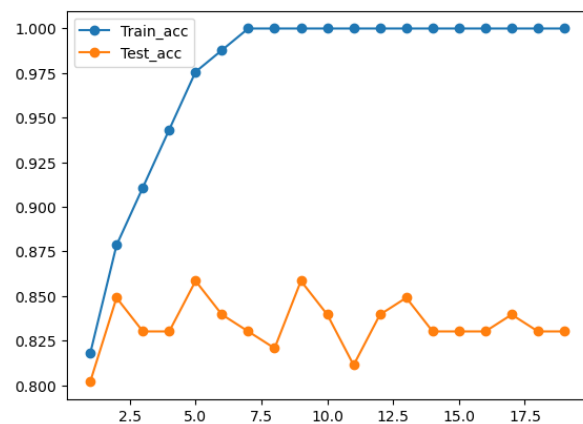
    #train 정확도와 test 정확도를 depth 개수에 따른 정확도 확인
    print('>%d, train: %.3f, test: %.3f' % (i, train_acc, test_acc))

>1, train: 0.818, test: 0.802
>2, train: 0.879, test: 0.849
>3, train: 0.911, test: 0.830
>4, train: 0.943, test: 0.830
>5, train: 0.976, test: 0.858
>6, train: 0.988, test: 0.840
>7, train: 1.000, test: 0.830
>8, train: 1.000, test: 0.821
>9, train: 1.000, test: 0.858
>10, train: 1.000, test: 0.840
>11, train: 1.000, test: 0.840
>12, train: 1.000, test: 0.840
>13, train: 1.000, test: 0.849
>14, train: 1.000, test: 0.830
>15, train: 1.000, test: 0.830
>16, train: 1.000, test: 0.830
>17, train: 1.000, test: 0.840
>18, train: 1.000, test: 0.830
>19, train: 1.000, test: 0.830
```

```
[86]: #그래프 그리기
```

```
from matplotlib import pyplot

pyplot.plot(range(1,20), train_scores, '-o', label='Train_acc')
pyplot.plot(range(1,20), test_scores, '-o', label='Test_acc')
pyplot.legend()
pyplot.show()
```



```
[87]: # 그래프만으로는 최적의 깊이를 가능하기 어려움
#test_scores에서 최고 점수의 인덱스를 찾아 최적의 깊이 확인
optimal_depth = np.argmax(test_scores) + 1 # 0부터 시작하므로 +1
optimal_test_acc = max(test_scores) # 최고 테스트 정확도

print(f"최적의 깊이는: {optimal_depth}")

최적의 깊이는: 5
```

```
[162]: # 의사결정나무모델에 SMOTE 적용한 데이터 적합(fitting)

from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=0, max_depth=5) #max_depth 5로 지정
tree.fit(X_train_balanced, y_train_balanced)
```

```
[162]: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=5, random_state=0)
```

```
[164]: #모델 성능 평가
temp_y_pred = tree.predict(X_test)
#Training값을 X_test에서 넣음
print('예측값\n', temp_y_pred)
print('실제값\n', y_test)

예측값
[0 0 0 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 0 1 0 0 1 0 0 0 0
 0 0 0 1 0 0 0 0 0 0 1 1 0 1 0 0 0 0 1 0 0 1 0 0 1 0 1 0 0 0 1 0 0 0 0
 0 0 1 0 1 1 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0]
실제값
[0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0
 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1
 0 1 1 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0]
```

```
[166]: # 정확도 계산
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix

temp_acc = accuracy_score(y_test, temp_y_pred)
# y_test와 temp_y_pred로 예측한 값을 적용하여 temp_acc를 추출함.

#원핫인코딩, 상관계수, 의사결정나무 깊이 최적화했을 때의 정확도
print('정확도: ', format(temp_acc))

정확도: 0.8113207547169812
```

```
[168]: #60211356 장유진
```

```
[170]: tree.predict_proba(X_test)[: ,1]
```

```
[170]: array([[0.26666667, 0.26666667, 0.
, 0.
, 1.
,
0.
, 0.
, 1.
, 0.
, 0.75
,
0.26666667, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 1.
, 0.95172414, 0.26666667, 0.
,
0.
, 0.95172414, 0.95172414, 0.
, 0.95172414,
0.
, 0.
, 1.
, 0.
, 0.26666667,
0.26666667, 0.
, 0.26666667, 0.
, 0.
,
0.95172414, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 1.
, 0.95172414, 0.26666667,
0.75
, 0.
, 0.
, 0.
, 0.
,
1.
, 0.
, 0.
, 0.75
, 0.
,
0.
, 0.95172414, 0.
, 0.95172414, 0.
,
0.75
, 0.
, 0.
, 0.
, 0.95172414,
0.26666667, 0.
, 0.
, 0.
, 0.
,
0.
, 0.95172414, 0.
, 0.95172414, 0.75
,
0.
, 0.
, 0.95172414, 0.
, 0.
,
0.
, 0.75
, 0.26666667, 1.
, 0.
,
0.
, 0.
, 0.75
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.26666667,
0.
, 0.
, 0.
, 0.95172414, 0.95172414,
0.
])
```

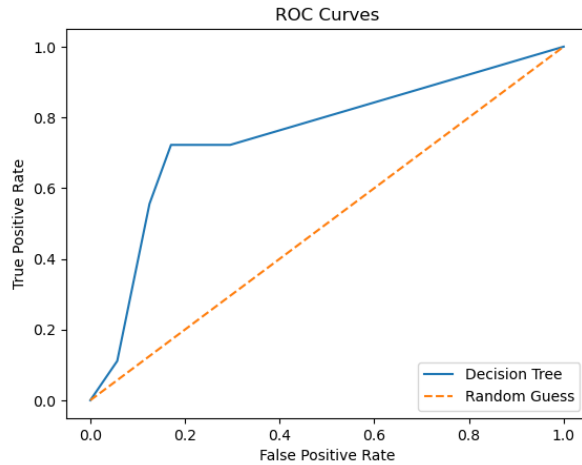
```
[172]: from sklearn.metrics import roc_curve
```

```
fpr, tpr, thresholds = roc_curve(y_test, tree.predict_proba(X_test)[:,1])
```

```
[174]: #ROC curve 그리기
```

```
plt.plot(fpr, tpr, '-', ms=2, label="Decision Tree")  
plt.plot([0,1], [0,1], '--', label="Random Guess")
```

```
plt.xlabel('False Positive Rate')  
plt.ylabel('True Positive Rate')  
plt.title('ROC Curves')  
plt.legend(loc="lower right")  
plt.show()
```



```
[176]: #AUC 계산
```

```
from sklearn import metrics  
roc_auc = metrics.auc(fpr, tpr)  
print('Area Under Curve : %0.2f' % roc_auc)
```

```
Area Under Curve : 0.75
```

```
[96]:
```

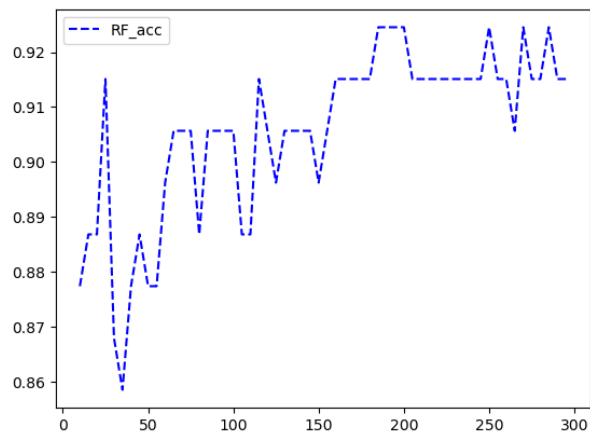
```
scores = []  
for i in range(10,300,5):  
    RF = RandomForestClassifier(n_estimators=i, random_state=0)  
    RF.fit(X_train, y_train)  
  
    att = RF.predict(X_test)  
    acc = accuracy_score(y_test, att)  
    scores.append(acc)  
  
print('>%d, acc: %.3f:' % (i, acc))
```

```
>100, acc: 0.906:  
>105, acc: 0.887:  
>110, acc: 0.887:  
>115, acc: 0.915:  
>120, acc: 0.906:  
>125, acc: 0.896:  
>130, acc: 0.906:  
>135, acc: 0.906:  
>140, acc: 0.906:  
>145, acc: 0.906:  
>150, acc: 0.896:  
>155, acc: 0.906:  
>160, acc: 0.915:  
>165, acc: 0.915:  
>170, acc: 0.915:  
>175, acc: 0.915:  
>180, acc: 0.915:  
>185, acc: 0.925:
```

```
[97]: import matplotlib.pyplot as plt
```

```
plt.plot(range(10, 10 + 5 * len(scores), 5), scores, 'b--', label='RF_acc')  
plt.legend()
```

[97]: <matplotlib.legend.Legend at 0x278a3da7d40>



```
[98]: # scores에서 가장 높은 정확도의 인덱스(최적의 나무 개수 확인)
      optimal_trees = np.argmax(scores) * 5 + 10 # i가 10부터 시작하고, 5씩 증가했으므로 계산
      optimal_acc = max(scores) # 최고 정확도

      print(f"최적의 나무 개수: {optimal_trees}, 정확도: {optimal_acc:.3f}")
      최적의 나무 개수: 185, 정확도: 0.925
```

```
[148]: # 의사결정나무모델에 데이터 적합(fitting)
      # 랜덤 포레스트
      # n_estimators = 185로 새로 적합

      from sklearn.ensemble import RandomForestClassifier

      RF = RandomForestClassifier(n_estimators=185, random_state=0)
      #n_estimators: 사용할 tree의 갯수.
```

```
[150]: # 모델 적합
      RF.fit(X_train, y_train)
```

```
[150]: + RandomForestClassifier
      RandomForestClassifier(n_estimators=185, random_state=0)
```

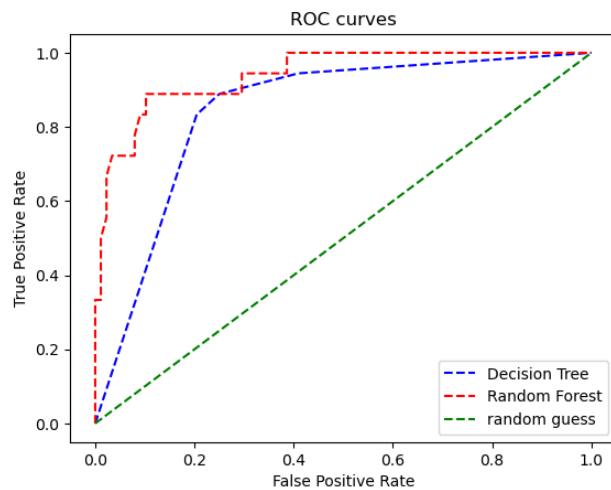
```
[152]: from sklearn.metrics import roc_curve

      fpr1, tpr1, thresholds1 = roc_curve(y_test, tree.predict_proba(X_test)[:,:1])
      fpr2, tpr2, thresholds2 = roc_curve(y_test, RF.predict_proba(X_test)[:,:1])
```

```
[154]: plt.plot(fpr1, tpr1, 'b--', label = 'Decision Tree')
      plt.plot(fpr2, tpr2, 'r--', label = 'Random Forest')
      plt.plot([0,1], [0,1], 'g--', label = 'random guess')

      plt.title('ROC curves')
      plt.xlabel('False Positive Rate')
      plt.ylabel('True Positive Rate')
      plt.legend(loc = 'lower right')
```

[154]: <matplotlib.legend.Legend at 0x278a48dbce0>

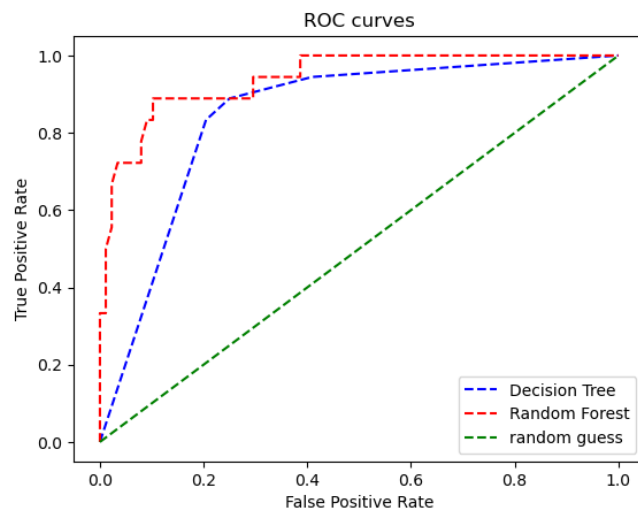


```
[156]: roc_auc_DT = metrics.auc(fpr1, tpr1)
roc_auc_RF = metrics.auc(fpr2, tpr2)
```

```
[158]: # 연립해 AUC 값 출력
plt.plot(fpr1, tpr1, 'b--', label = 'Decision Tree (area = %0.2f)' % roc_auc_DT)
plt.plot(fpr2, tpr2, 'r--', label = 'Random Forest (area = %0.2f)' % roc_auc_RF)
plt.plot([0,1], [0,1], 'g--', label = 'random guess')

plt.title('ROC curves')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc = 'lower right')
```

[154]: <matplotlib.legend.Legend at 0x278a48dbce0>

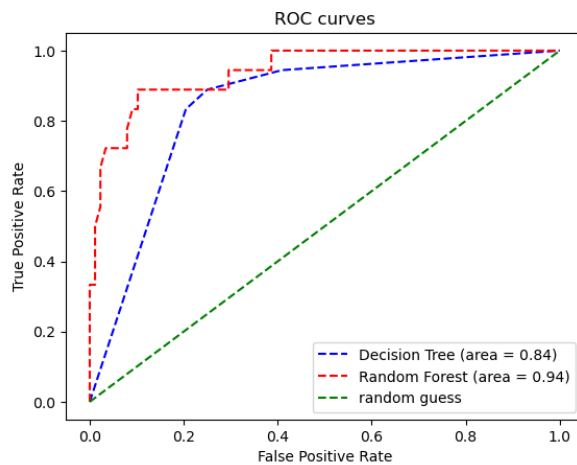


```
[156]: roc_auc_DT = metrics.auc(fpr1, tpr1)
roc_auc_RF = metrics.auc(fpr2, tpr2)
```

```
[158]: # 연립해 AUC 값 출력
plt.plot(fpr1, tpr1, 'b--', label = 'Decision Tree (area = %0.2f)' % roc_auc_DT)
plt.plot(fpr2, tpr2, 'r--', label = 'Random Forest (area = %0.2f)' % roc_auc_RF)
plt.plot([0,1], [0,1], 'g--', label = 'random guess')
```

```
plt.title('ROC curves')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc = 'lower right')
```

[158]: <matplotlib.legend.Legend at 0x278a4735d60>



```
[160]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
# 성능 평가
temp_y_pred_rf = RF.predict(X_test)

print('accuracy: ', accuracy_score(y_test, temp_y_pred_rf))
print('precision: ', precision_score(y_test, temp_y_pred_rf))
print('recall: ', recall_score(y_test, temp_y_pred_rf))
print('f1: ', f1_score(y_test, temp_y_pred_rf))
```

```
accuracy: 0.9245283018867925
precision: 0.8571428571428571
recall: 0.6666666666666666
f1: 0.75
```

[106]: #60211356 공유진

후기

Random Forest 모델에 대해 학습하였다. 의사결정나무와 유사하여 받아들이는 데에 어려움이 적었다. 의사결정나무에서는 깊이를 최적화하였는데, Random Forest에서는 나무의 개수를 최적화하는 것이 기억에 남는다. 최적의 나무 개수를 시각적으로 확인하기 어려워 역시나 구글링을 통해 최적화된 값을 구하였다.

의사결정나무 모델과 비교하면 정확도가 상승하였다. 의사결정나무모델의 정확도는 약 0.811이었으나, Random Forest 모델의 정확도는 약 0.925이다. 이 모델을 효율적으로 사용하는 법을 익혀야겠다고 생각했다.